

Nomina SQL Helper Functions

A practical guide for adding name families faster, with safer reuse of existing nodes

This guide explains how to use the new helper layer on top of **sql_generator.py**. The helper functions do not replace your SQL builder. They simply remove repeated boilerplate when adding names, word nodes, edges, and lineages.

1. File setup

Keep the files separated. This avoids copying the builder and helper code into every new batch.

```
nomina_project/
  sql_generator.py      # your main builder: Language, Source, ConceptNode,
                        WordNode, NameNode, Edge, Lineage, NominaSQLBuilder
  sql_helpers.py        # helper functions: add_direct_name, add_name_variant,
                        add_compound_name, etc.
  batch_moon_04.py      # one batch script you run to generate SQL
```

Every new batch file should start with:

```
from sql_generator import *
from sql_helpers import *
```

You should not paste the helper functions after the original builder in every batch. Put them once in **sql_helpers.py**, then import them.

2. Why this is faster

Previously, a simple name required repeated code for the word, name, concept edge, word-to-name edge, and primary lineage. With helpers, you still define the word and name carefully, but the helper creates the repeated graph structure for you.

Helper	Use it when	Creates
add_direct_name	A name comes directly from one word or word-form.	word node, name node, concept -> word edge(s), word -> name edge, primary lineage
add_word_chain_name	A name passes through a chain of words or language forms.	all word nodes, chain edges, final name node, primary lineage
add_name_variant	A name is a spelling/form variant of a base name already registered in the builder.	variant name node, variant_of edge, primary lineage
add_compound_name	A name is built from multiple word parts, such as moon + child or cold + ray.	part words, compound word, name, element edges, concept edges, primary lineage
require_existing_node	A previous batch already created a node and the current batch only needs to refer to it.	a variable handle for that existing node, with an error if it is missing

3. The standard workflow

- Create a new batch file, for example **batch_star_02.py**.
- Import from **sql_generator** and **sql_helpers**.
- Create the builder: **b = NominaSQLBuilder()**.
- Call **b.include_edge_types()**.
- Ensure the languages and sources needed for the batch.
- Ensure concept nodes, or require existing nodes where you deliberately do not want to redefine them.

- Add each name using the correct helper pattern.
- Render the SQL and run the generated .sql file in Supabase.

```
def build_example_family_sql() -> str:
    b = NominaSQLBuilder()
    b.include_edge_types()

    ensure_languages(b, [
        Language(slug="welsh", name="Welsh", language_family="Celtic /
            Indo-European"),
    ])

    b.ensure_sources(
        group_name="example_family",
        sources=[
            Source(
                title="Example Source - Seren",
                author="Example Source",
                source_type="name dictionary",
                url="https://example.com/seren",
                citation_text="Example Source, 'Seren'.",
                notes="Used for Seren from Welsh seren, meaning star.",
            ),
        ],
    )

    ensure_concepts(b, [
        ConceptNode(
            slug="star-concept",
            label="star",
            canonical_key="star",
            description_short="The idea of a star or celestial point of light.",
            description="The idea of a star or celestial point of light.",
            source_group="example_family",
            lookup_slugs=["star"],
            lookup_labels=["star"],
            lookup_canonical_keys=["star"],
        )
    ])

    # helper calls go here

    return b.render()
```

4. Pattern A: direct word -> name

Use **add_direct_name** when the etymology is clean and direct. Examples: Welsh *seren* -> Seren, Arabic *qamar* -> Qamar, Latin *luna* -> Luna.

```
add_direct_name(
  b,
  word=WordNode(
    slug="welsh-seren",
    label="seren",
    canonical_key="seren",
    description_short="Welsh word/name element meaning star.",
    language_slug="welsh",
    display_text="seren",
    original_script="seren",
    transliteration="seren",
    word_type="noun / name element",
    literal_meaning="star",
    grammar_notes="Welsh noun meaning star.",
    additional_notes="Seren is a modern Welsh given name formed directly from
      the Welsh star word.",
    source_group="star_family_02",
  ),
  name=NameNode(
    slug="seren",
    label="Seren",
    canonical_key="seren",
    description_short="Welsh given name meaning star.",
    display_name="Seren",
    original_script="Seren",
    transliteration="Seren",
    primary_language_slug="welsh",
    gender_usage="feminine given name; also occasionally masculine",
    short_summary="Seren is a Welsh given name meaning star.",
    long_summary="Seren comes directly from Welsh seren, meaning star.",
    literary_notes=None,
    cultural_notes="This is mainly a first/given name and has become a
      familiar modern Welsh name.",
    pronunciation_notes="Usually pronounced SEH-ren or SEH-run.",
    certainty_notes="Good certainty for the Welsh star meaning.",
    source_group="star_family_02",
  ),
  concept_edges=[
    MeaningEdge("star-concept", "Welsh seren means star.", 0.94),
  ],
  word_to_name_explanation="Welsh seren forms the name Seren.",
  word_to_name_certainty=0.92,
  lineage_summary="Seren derives directly from Welsh seren, meaning star.",
  lineage_certainty=0.92,
  source_group="star_family_02",
)
```

This one call creates the word node, name node, concept edge, element edge, and lineage.

5. Pattern B: word chain -> name

Use **add_word_chain_name** when a name passes through several language stages or intermediate word forms. Examples: Latin *caelum* -> Spanish *cielo* -> Cielo, or Persian *yasamin* -> Arabic *yasmin* -> English *jasmine* -> Jasmine.

```
add_word_chain_name(
  b,
  words=[latin-caelum, spanish-cielo],
  name=cielo_name,
  concept_edges=[
    MeaningEdge("sky-heavens", "Latin caelum means sky or heaven.", 0.94),
  ],
  chain_edges=[
    ChainEdge(
      from_slug="latin-caelum",
      to_slug="spanish-cielo",
      edge_type_code="evolves_to",
      explanation="Spanish cielo ultimately belongs to the Latin caelum
        sky/heaven word family.",
    )
  ]
)
```

```

        certainty=0.82,
    )
],
final_word_to_name_explanation="Spanish cielo forms the name Cielo.",
final_word_to_name_certainty=0.92,
lineage_summary="Cielo derives from Spanish cielo, meaning sky or heaven.",
lineage_certainty=0.90,
source_group="sky_heavens_family_01",
)

```

The number of **chain_edges** must be exactly one less than the number of words. If there are two words, use one chain edge. If there are three words, use two chain edges.

6. Pattern C: variant name

Use **add_name_variant** when the name is a variant or spelling form of a base name. The base name must already be registered in the current builder. If it was created in an earlier SQL batch, use **require_existing_node** first.

```

require_existing_node(
    b,
    node_type="name",
    slug="lucia",
    label="Lucia",
    canonical_key="lucia",
)

add_name_variant(
    b,
    variant=NameNode(
        slug="lucy",
        label="Lucy",
        canonical_key="lucy",
        description_short="English feminine given name, the English form of
            Lucia.",
        display_name="Lucy",
        original_script="Lucy",
        transliteration="Lucy",
        primary_language_slug="english",
        gender_usage="feminine given name",
        short_summary="Lucy is an English feminine given name, the English form of
            Lucia.",
        long_summary="Lucy is the English form of Lucia and belongs to the Latin
            lux light-name family.",
        literary_notes=None,
        cultural_notes="This is mainly a first/given name and is associated with
            Saint Lucy.",
        pronunciation_notes="Usually pronounced L00-see.",
        certainty_notes="Good certainty for Lucy as the English form of Lucia.",
        source_group="lucy_patch",
    ),
    base_slug="lucia",
    variant_explanation="Lucy is the English form of Lucia.",
    variant_certainty=0.92,
    lineage_path=[
        ["lamp-light"],
        ["latin-lux"],
        ["latin-lucia"],
        ["lucia"],
        ["lucy"],
    ],
    lineage_summary="Lucy is the English form of Lucia, ultimately associated with
        Latin lux, light.",
    lineage_certainty=0.90,
    source_group="lucy_patch",
)

```

Use a custom lineage path when you want the variant to inherit the full route of the base name rather than showing only base -> variant.

7. Pattern D: compound name

Use **add_compound_name** when the name is built from word parts. Examples: Tsukiko = moon + child, Pankaj = mud + born, Himanshu = cold + ray.

```
add_compound_name(  
  b,  
  parts=[  
    CompoundPart(  
      word=WordNode(  
        slug="japanese-tsuki",  
        label="tsuki / moon character",  
        canonical_key="tsuki",  
        description_short="Japanese word/name element meaning moon or month.",  
        language_slug="japanese",  
        display_text="moon character",  
        original_script="moon character",  
        transliteration="tsuki",  
        word_type="noun / name element",  
        literal_meaning="moon, month",  
        grammar_notes="Japanese moon character can be read tsuki and means moon  
          or month.",  
        additional_notes="Exact name meaning depends on the kanji used.",  
        source_group="mixed_moon_family_03",  
      ),  
      concept_edges=[MeaningEdge("moon-concept", "Japanese tsuki means moon or  
        month.", 0.88)],  
      explanation_to_compound="Japanese tsuki supplies the moon element in  
        Tsukiko.",  
      certainty_to_compound=0.88,  
    ),  
    CompoundPart(  
      word=WordNode(  
        slug="japanese-ko-child",  
        label="ko / child character",  
        canonical_key="ko child",  
        description_short="Japanese name element ko meaning child.",  
        language_slug="japanese",  
        display_text="child character",  
        original_script="child character",  
        transliteration="ko",  
        word_type="name element / suffix",  
        literal_meaning="child",  
        grammar_notes="Japanese name element often used in feminine names.",  
        additional_notes="This is the child element in names such as Tsukiko.",  
        source_group="mixed_moon_family_03",  
      ),  
      concept_edges=[MeaningEdge("child", "Japanese ko means child.", 0.90)],  
      explanation_to_compound="Japanese ko supplies the child element in  
        Tsukiko.",  
      certainty_to_compound=0.90,  
    ),  
  ],  
  compound_word=WordNode(  
    slug="japanese-tsukiko",  
    label="Tsukiko",  
    canonical_key="tsukiko",  
    description_short="Japanese compound name form meaning moon child.",  
    language_slug="japanese",  
    display_text="Tsukiko",  
    original_script="Tsukiko",  
    transliteration="Tsukiko",  
    word_type="compound name form",  
    literal_meaning="moon child",  
    grammar_notes="Name form from tsuki, moon, and ko, child.",  
    additional_notes="The moon-child meaning depends on the written characters.",  
    source_group="mixed_moon_family_03",  
  ),  
  name=tsukiko_name,  
  compound_meaning_edges=[  
    MeaningEdge("moon-concept", "Tsukiko carries moon imagery through tsuki.",  
      0.86),  
    MeaningEdge("child", "Tsukiko carries the child element through ko.", 0.88),  
  ],  
  compound_to_name_explanation="Japanese Tsukiko forms the given name Tsukiko.",  
  compound_to_name_certainty=0.88,  
  lineage_summary="Tsukiko can be written from Japanese tsuki, moon, and ko, child.",  
)
```

```

lineage_certainty=0.88,
source_group="mixed_moon_family_03",
)

```

For Japanese and Chinese names, be especially careful in the notes: the meaning depends on the written characters. Do not imply that every person with that romanised name has the same meaning.

8. Reusing existing nodes safely

The builder already reuses existing nodes by looking up slug, display label, and canonical key. It does not overwrite existing node details unless you explicitly set update flags. When you need a node created in an earlier batch but do not want to define it again, use **require_existing_node**.

```

require_existing_node(
    b,
    node_type="word",
    slug="turkish-ay",
    label="ay",
    canonical_key="ay",
    lookup_labels=["ay"],
    lookup_canonical_keys=["ay"],
)

```

This registers **turkish-ay** inside the current builder. If the database does not contain it, the generated SQL raises an exception instead of silently creating a duplicate or broken lineage.

9. Safety rules

- Existing concept, word, and name detail rows are not overwritten by default.
- Only set **update_existing_node=True** or **update_existing_detail=True** when you intentionally want to revise an existing node or detail row.
- Avoid creating duplicate concept nodes. If unsure, ask which concepts already exist first.
- Keep concept nodes simple: **moon**, **star**, **child**, **cold**, **grace**. Avoid vague combined nodes like pride/grace unless both meanings are truly inseparable.
- Do not write name notes that say 'the graph uses...' or 'this graph connects...'. Name notes should describe the name itself.
- Use cautious certainty notes when an etymology is debated, character-dependent, or source-dependent.

10. Common errors and fixes

Helper	Use it when	Creates
Base node not registered	You call <code>add_name_variant</code> with <code>base_slug='lucia'</code> , but <code>lucia</code> was not ensured or required in this builder.	Call <code>require_existing_node(...)</code> before <code>add_name_variant</code> .
Lineage validation error	Adjacent lineage groups have no explicit edge between them.	Add the missing Edge through a helper argument, or simplify the lineage path.
Duplicate edge constraint	The database allows only one edge per node pair regardless of direction/type.	Your <code>ensure_edge</code> deletes both directions before inserting. Keep using the corrected <code>ensure_edge</code> method.
Accidental concept duplicate	A concept exists under another slug, for example <code>moon</code> vs <code>moon-concept</code> .	Use <code>lookup_slugs</code> , <code>lookup_labels</code> , and <code>lookup_canonical_keys</code> so the builder reuses the existing node.
Japanese/Chinese overclaim	Romanised name can be written with several characters.	State that the meaning applies when written with the specific character.

11. Compact batch template

```
from sql_generator import *
from sql_helpers import *

def build_new_family_sql() -> str:
    b = NominaSQLBuilder()
    b.include_edge_types()

    ensure_languages(b, [
        Language(slug="example-language", name="Example Language"),
    ])

    b.ensure_sources(
        group_name="new_family_01",
        sources=[
            Source(
                title="Source Title",
                author="Source Author",
                source_type="name dictionary",
                url="https://example.com",
                citation_text="Source Author, 'Entry Title'.",
                notes="Used for the name meaning and route.",
            ),
        ],
    )

    ensure_concepts(b, [
        ConceptNode(
            slug="example-concept",
            label="example concept",
            canonical_key="example concept",
            description_short="Short concept description.",
            description="Longer concept description.",
            source_group="new_family_01",
            lookup_slugs=["example"],
            lookup_labels=["example concept"],
            lookup_canonical_keys=["example concept"],
        ),
    ])

    # add_direct_name(...)
    # add_word_chain_name(...)
    # add_name_variant(...)
    # add_compound_name(...)

    return b.render()

if __name__ == "__main__":
    sql = build_new_family_sql()
    with open("nomina_batch_new_family_01.sql", "w", encoding="utf-8") as f:
        f.write(sql)
    print("Wrote nomina_batch_new_family_01.sql")
```

12. Quick decision guide

Helper	Use it when	Creates
One word gives the name	Luna, Qamar, Seren, Hoshi	Use <code>add_direct_name</code>
Several word stages	Latin caelum -> Spanish cielo -> Cielo	Use <code>add_word_chain_name</code>
Spelling/form variant	Noor -> Nur, Lucy -> Lucia	Use <code>add_name_variant</code>
Two or more word parts	Tsukiko = moon + child, Himanshu = cold + ray	Use <code>add_compound_name</code>
Node already exists	turkish-ay, latin-lux, lucia	Use <code>require_existing_node</code> if you only need a handle

Final reminder

The helper layer is there to reduce repetitive code, not to reduce judgement. Research the name, choose clean concept nodes, write notes about the name rather than the database, and keep uncertainty visible where sources disagree.